Web Automation using Selenium with Python

Capstone Project Presentation

Presented by: Sushant

Course : Wipro Python Automation Training

Date: 10th March 2026

Made with GAMMA

Problem Statement

- Automate the core user flows of the BrowserStack E-Commerce demo application.
- Ensure the tests are stable and do not fail due to modern web elements like dynamic dropdowns or loading animations.
- Build a framework that can handle multiple users logging in sequentially without caching issues.

Made with GAMMA

Project Objectives

Implement the Page Object Model (POM)

To keep locators and test scripts separate.

Apply Data-Driven Testing (DDT)

So we can test multiple scenarios without rewriting code.

Automate browser setup and teardown

Using Pytest fixtures.

Generate an easy-to-read HTML report

With screenshots for failed tests.

Made with GAMMA

Tools and Technologies Used

Programming Language

Python 3.11

Browser Automation

Selenium WebDriver

Test Runner

Pytest

Data Management

Pandas & OpenPyXL (for reading Excel files)

Parallel Execution

Pytest-xdist

Reporting

Pytest-HTML

Made with GAMMA

Automation Framework Architecture

Pages Folder

Contains the Base_page.py (common actions like clicking and typing) and specific pages (Login, Home, Checkout).

Tests Folder

Contains the Pytest test files and conftest.py (which handles opening and closing the browser).

Utilities Folder

Contains helper functions like the Excel reader and a custom logger.

Test Data

Uses a testdata.xlsx file to feed data into the checkout test.

Made with GAMMA

Project Workflow

Step 1

Pytest starts and triggers conftest.py to open the Chrome browser.

Step 2

Test scripts read data from our Excel file or from Pytest parameters.

Step 3

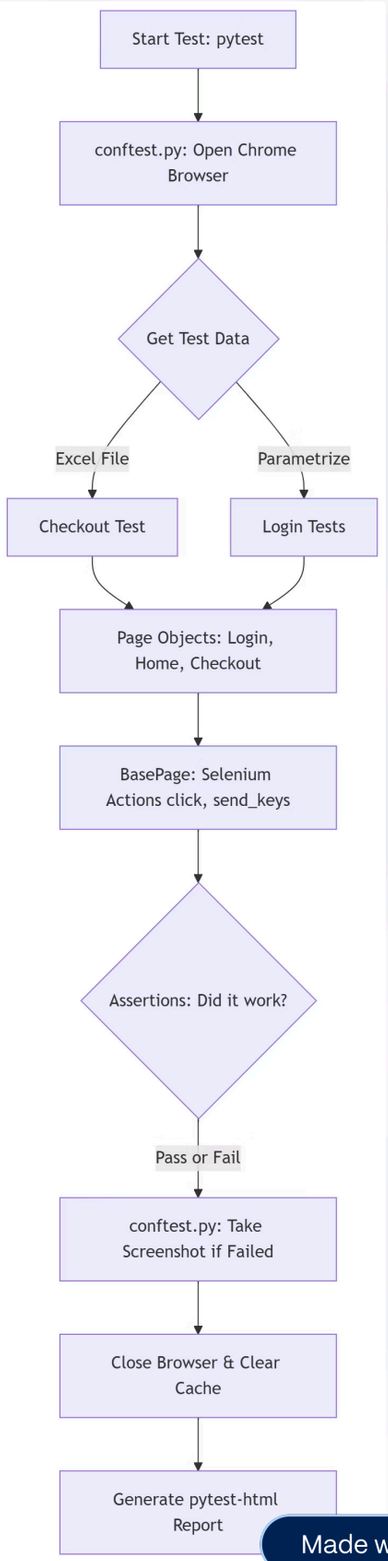
The test calls methods from the Page Objects to interact with the website.

Step 4

Assertions check if the test passed (e.g., verifying a success message).

Step 5

The browser clears its cookies and local storage, then closes safely.

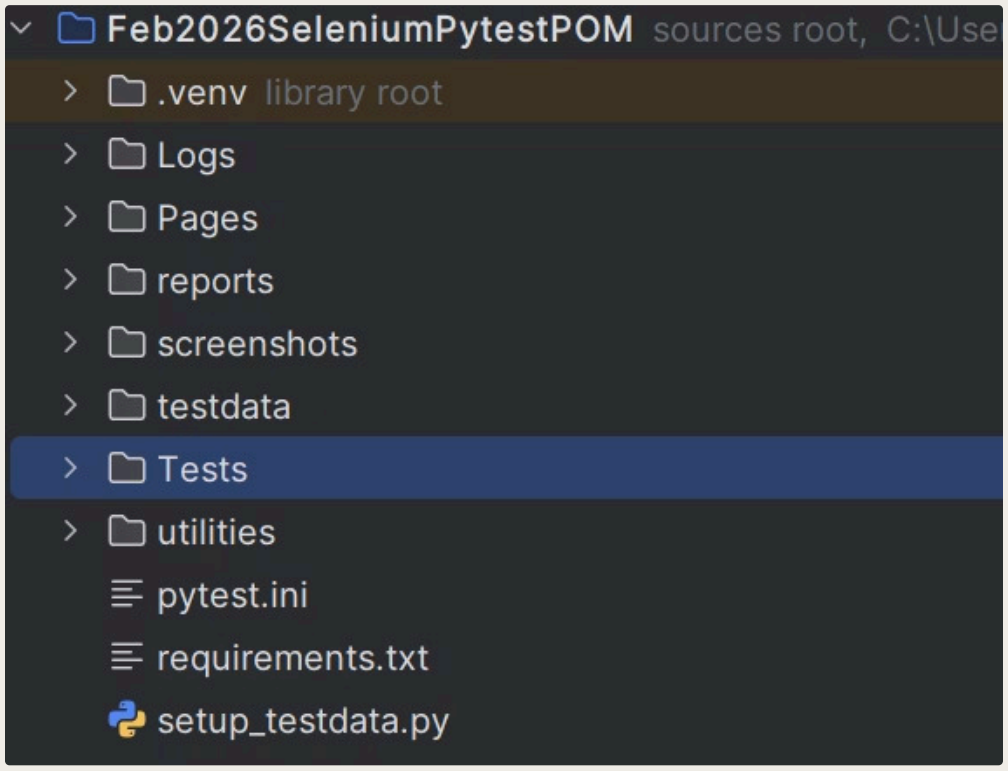


Made with GAMMA

Project Test Output – Screen shots

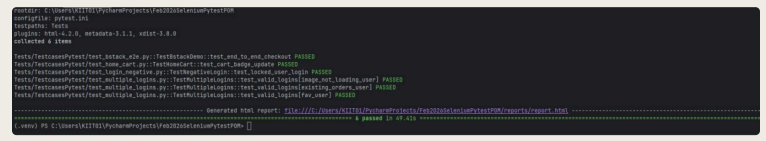
- ## Image 1

  PyCharm project folder structure showing Pages, Tests, and Utilities.



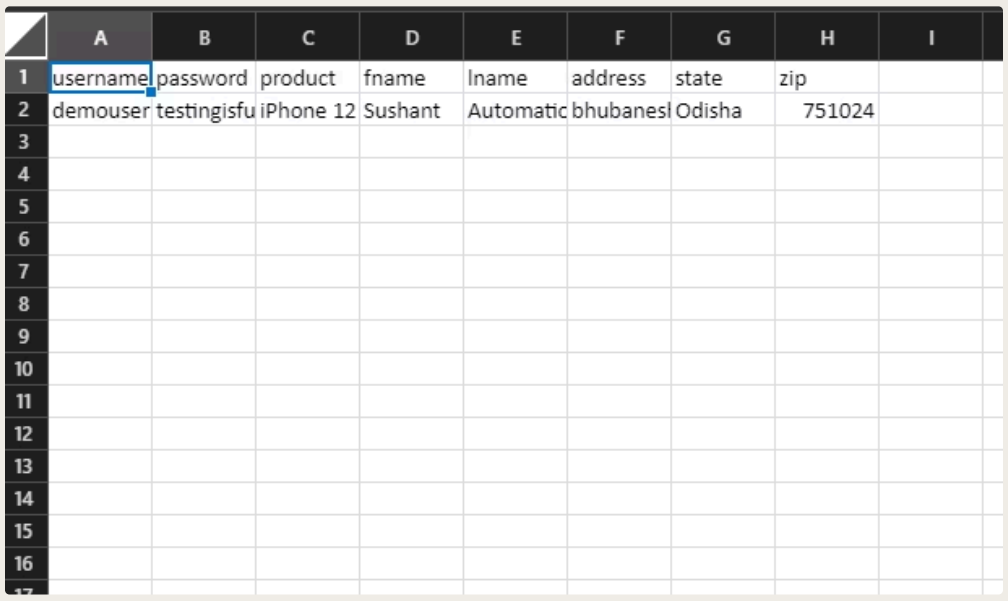
- ## Image 2

  Terminal showing the pytest -s command passing all 6 tests.



- ## Image 3

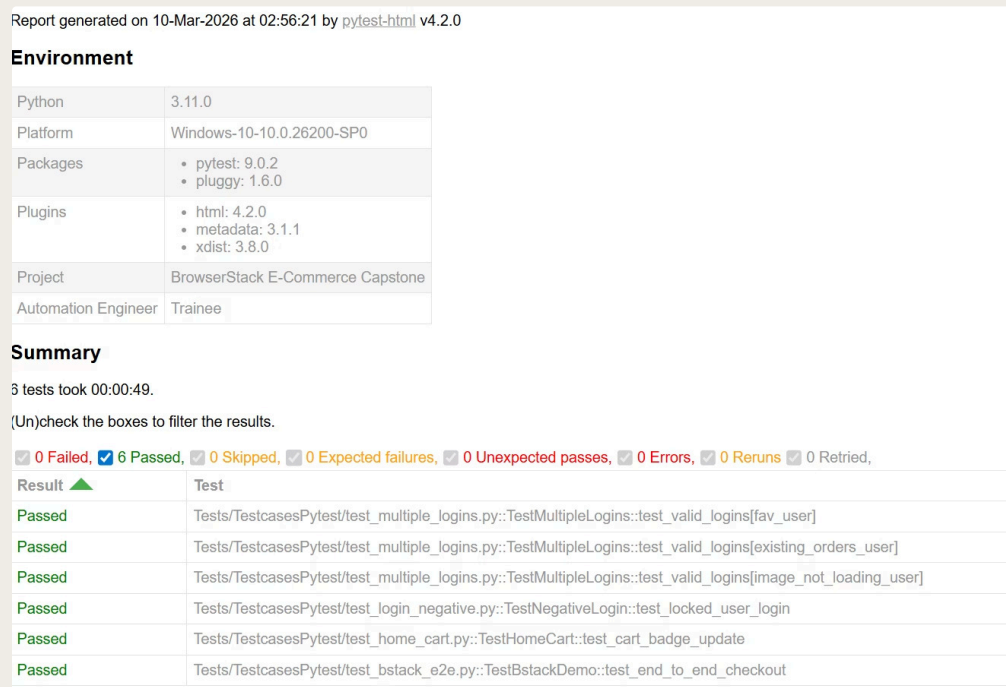
  The testdata.xlsx file showing the external test data.



	A	B	C	D	E	F	G	H	I
1	username	password	product	fname	lname	address	state	zip	
2	demouser	testingisfu	iPhone 12	Sushant	Automatic	bhubanesl	Odisha	751024	
3									
4									
5									
6									
7									
8									
9									
10									
11									
12									
13									
14									
15									
16									

- ## Image 4

  The generated report.html page showing the final test results.



Report generated on 10-Mar-2026 at 02:56:21 by pytest-html v4.2.0

Environment

Python	3.11.0
Platform	Windows-10-10.0.26200-SP0
Packages	• pytest: 9.0.2 • pluggy: 1.6.0
Plugins	• html: 4.2.0 • metadata: 3.1.1 • xdist: 3.8.0
Project	BrowserStack E-Commerce Capstone
Automation Engineer	Trainee

Summary

6 tests took 00:00:49.

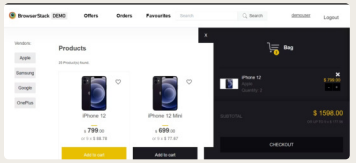
(Un)check the boxes to filter the results.

0 Failed, 6 Passed, 0 Skipped, 0 Expected failures, 0 Unexpected passes, 0 Errors, 0 Reruns 0 Retried,

Result	Test
Passed	Tests/TestcasesPytest/test_multiple_logins.py::TestMultipleLogins::test_valid_logins[fav_user]
Passed	Tests/TestcasesPytest/test_multiple_logins.py::TestMultipleLogins::test_valid_logins[existing_orders_user]
Passed	Tests/TestcasesPytest/test_multiple_logins.py::TestMultipleLogins::test_valid_logins[image_not_loading_user]
Passed	Tests/TestcasesPytest/test_login_negative.py::TestNegativeLogin::test_locked_user_login
Passed	Tests/TestcasesPytest/test_home_cart.py::TestHomeCart::test_cart_badge_update
Passed	Tests/TestcasesPytest/test_bstack_e2e.py::TestBstackDemo::test_end_to_end_checkout

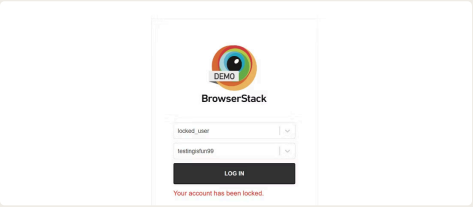
Made with GAMMA

Test Failed - screenshots



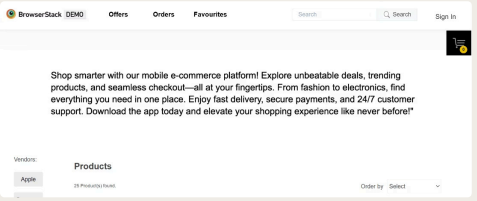
1

test_end_to_end_checkout_failed.png



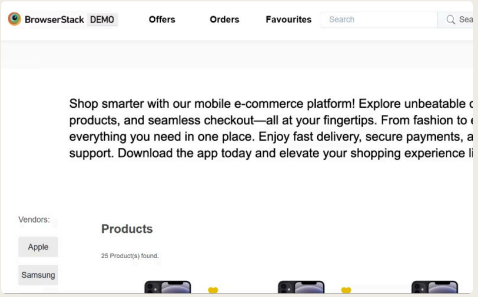
2

test_locked_user_login_failed.png



3

test_valid_logins[existing_orders_user]_failed.png



4

test_valid_logins[fav_user]_failed.png

Made with GAMMA

Project Output

Successfully automated an End-to-End (E2E) checkout flow using external Excel data.

Automated a UI validation test to check if the cart badge updates correctly.

Created a negative test to verify that locked-out users get the correct error message.

Set up parameterized tests to quickly check multiple user logins using a single script.

Automatically generated an HTML report logging the entire test run.

Made with GAMMA

Challenges Faced

Dynamic Element IDs

The login dropdown IDs kept changing randomly (e.g., react-select-2 to -4). I fixed this by updating my locators to use static CSS selectors (#username input).

Ghost Login Sessions

When looping through multiple users, the site remembered the previous user because of Local Storage. I fixed this by forcing the browser to clear Cookies and Local Storage between tests to guarantee a fresh start.

Made with GAMMA

Future Enhancements

Run the tests on different browsers

Like Firefox and Edge (Cross-Browser Testing).

Integrate the tests into a CI/CD pipeline

(like GitHub Actions) so they run automatically on every code push.

Run the tests in "Headless" mode

To make them execute faster in the background without opening a visible window.

Made with GAMMA

Conclusion & Learnings

- The Page Object Model makes maintaining code much easier; if the UI changes, I only have to update locators in one file.
- Pytest fixtures (`conftest.py`) are incredibly helpful for keeping setup and teardown code out of the main test scripts.
- Working with modern web apps requires extra attention to how the browser stores data (like local storage) between tests.

Made with GAMMA